

# Implementing Convolution

Algorithms and Deep Learning Context

---

Computer Vision & AI

# 1. Defining the Data

In code, images and kernels are just 2D NumPy arrays. Let's create a dummy image and a simple edge-detection kernel.

```
import numpy as np

# 5x5 Dummy Image
image = np.array([
    [10, 10, 10, 0, 0],
    [10, 10, 10, 0, 0],
    [10, 10, 10, 0, 0],
    [10, 10, 10, 0, 0],
    [10, 10, 10, 0, 0]
])

# 3x3 Vertical Edge Kernel
kernel = np.array([
    [1, 0, -1],
    [1, 0, -1],
    [1, 0, -1]
])
```

## 2. The Naive Approach (From Scratch)

How do we write the sliding window algorithm using pure syntax?  
(Note: This is Cross-Correlation, typical in ML).

```
def naive_convolve2d(img, kernel):
    img_h, img_w = img.shape
    k_h, k_w = kernel.shape

    # Calculate output dimensions (Assuming Stride=1, Pad=0)
    out_h = img_h - k_h + 1
    out_w = img_w - k_w + 1
    output = np.zeros((out_h, out_w))

    # Slide the window
    for y in range(out_h):
        for x in range(out_w):
            # Extract the current patch
            patch = img[y:y+k_h, x:x+k_w]
            # Element-wise multiplication and sum
            output[y, x] = np.sum(patch * kernel)

    return output
```

### 3. Running the Code

Let's see the result of our vertical edge detector.

```
result = naive_convolve2d(image, kernel)
print(result)
```

```
"""
```

```
Output:
```

```
[[30. 30.  0.]
 [30. 30.  0.]
 [30. 30.  0.]]
"""
```

- The high values (30) indicate exactly where the bright pixels transition to dark pixels (the vertical edge).

## 4. The Problem with Naive Loops

While writing for loops helps us understand the logic, it is computationally terrible for Computer Vision.

```
# A modern 4K image is 3840 x 2160 pixels.  
# A standard CNN layer might have 64 filters.  
  
# Nested loops in Python are slow because of interpreter  
  overhead.  
# Big O Complexity: O(Image_Height * Image_Width *  
  Kernel_Area)  
  
# We need Vectorization!
```

- Real-world software engineering requires C/C++ backend integration (like NumPy/SciPy) or GPU acceleration.

## 5. Vectorized Approach (SciPy)

Instead of writing loops, we use optimized libraries. SciPy uses highly optimized C code under the hood.

```
from scipy.signal import convolve2d

# The 'valid' mode means no padding, same as our naive
# approach.
# SciPy does mathematically true convolution (it flips the
# kernel).
fast_result = convolve2d(image, kernel, mode='valid')

# For ML (Cross-Correlation without flip), we use
# correlate2d
from scipy.signal import correlate2d
ml_result = correlate2d(image, kernel, mode='valid')
```

## 6. Handling Padding in Code

To keep the output size the same as the input, we pad the array with zeros before convolving.

```
def pad_image(img, pad_size):
    # np.pad adds 'pad_size' zeros to all sides
    return np.pad(img, pad_width=pad_size, mode='constant',
                   constant_values=0)

padded_img = pad_image(image, pad_size=1)
# padded_img is now 7x7

# Output size = (7 - 3 + 1) = 5
# Output is now 5x5, matching original image!
padded_result = correlate2d(padded_img, kernel, mode='valid',
                             )
```

## 7. Deep Learning Context (PyTorch)

In AI, we don't hardcode the kernel values (like our 1, 0, -1). We let the network **learn** them.

```
import torch
import torch.nn as nn

# Define a 2D Convolutional Layer
# 1 input channel (Grayscale), 16 output filters, 3x3 kernel
conv_layer = nn.Conv2d(in_channels=1, out_channels=16,
                        kernel_size=3, padding=1, stride=1)

# Convert our image to a PyTorch Tensor: (Batch, Channel, H, W)
tensor_img = torch.tensor(image, dtype=torch.float32).view
(1, 1, 5, 5)

# Forward pass (The GPU does the math instantly)
output_tensor = conv_layer(tensor_img)
```



## 8. Software Engineering Takeaways

### Bridging Theory and Practice:

- **Syntax:** Arrays, nested loops, indices.
- **Semantics:** Extracting visual features, translation invariance.
- **Architecture:** Never write manual loops for tensor math in production. Use BLAS/cuDNN optimized APIs (NumPy, PyTorch).
- **Memory Management:** Convolutions take less RAM for weights than Dense layers, but feature maps can consume massive VRAM during backpropagation.